

MULTI-STATE RWKV ONLINE MEMORY: A MECHANISM AND SYSTEMS STUDY

OPENAI

ABSTRACT. We study multi-state RWKV online memory as a compact recurrent path for hybrid-attention language models. The deployed adapter maintains several FP32 matrix states in a low-rank feature space, assigns writes by fixed context chunks, reads all states with cosine-routed softmax weights before writing, and injects the read through trainable low-rank attention corrections while the backbone remains frozen. We separately audit a synthetic proxy that uses direct key/value features, fixed recurrence coefficients, externally supplied blocks, and known block-index reads; it isolates recurrence and state allocation rather than the deployed writer/router. In that proxy, multi-state RWKV reaches perfect recall in all three reported configurations, while the one-state comparator scores 0.797, 0.682, and 0.626. Across 15 identical-block cells, including four perfect-boundary ties, the RWKV-7 update wins 11 against a linear state and loses none. A surprise-gated exact cache improves the hardest low-capacity proxy cell from 0.669 to 0.880. A six-layer rank-8 Gemma4 E4B adapter reaches 14/20 on a no-rule Tau2 screen, versus 4/20 for the accepted frozen-base setup, but this result is not yet confirmed at larger scale. Later scene-memory controls show write and nonzero-state effects without a reliable advantage for the correct history over a donor history. The evidence therefore supports useful proxy recurrence behavior and a reproducible runtime prototype, not yet a validated general-purpose learned memory system.

1. SCOPE AND RESEARCH QUESTIONS

The repository contains two related but logically separate studies [1]. The first is a training-free proxy study on synthetic key/value streams: it asks whether several RWKV-7 matrix states preserve associative information better than one state, and whether the recurrent update is robust when externally supplied state boundaries are imperfect. The second attaches a learned online-memory adapter to a frozen Gemma4 E4B language model and asks whether the production mechanism can affect agent behavior and scene-history prediction. Synthetic recall is evidence about a specialized recurrence and allocation setup; it is not an end-to-end test of the adapter or evidence that a language model has learned useful natural-language memory.

We organize the evidence around four questions:

- (1) How does multi-state allocation compare with a single RWKV state in controlled recall?
- (2) Under the same boundaries, is the RWKV-7 state update more robust than a linear associative state?
- (3) Can a small learned adapter improve a frozen model on an application screen?
- (4) Do causal controls show that predictions depend on the identity of the stored history, rather than merely the presence of nonzero state?

All reported values below are transcribed from the repository reports and code snapshot listed in Section 7; no new training or benchmark run is claimed here.

2. ONLINE-MEMORY MECHANISM

2.1. Persistent multi-state recurrence. For each wrapped attention layer, the model projects a normalized low-rank source into time-shifted RWKV-7 features $r_t, w_t, k_t, v_t, a_t, b_t, g_t$, following the RWKV-7 state-evolution parameterization [2]. For batch element b and state head h , the persistent memory is

$$S \in \mathbb{R}^{B \times H_s \times M \times d \times d},$$

where M is the number of slots and d is the adapter rank. The recurrent matrices are allocated in FP32 and initialized to zero. A token at persistent per-session memory position p_t selects a write slot by a fixed cyclic chunk schedule,

$$j_t = \left\lfloor \frac{p_t}{C} \right\rfloor \bmod M,$$

with chunk length C . Thus write boundaries are not learned; routing adaptivity is on the read side.

Every slot is read before the selected slot is updated. Suppressing batch and head indices,

$$\begin{aligned}
 (1) \quad & u_{s,t} = S_{s,t} r_t, \\
 (2) \quad & \alpha_{s,t} = \text{softmax}_s(\cos(u_{s,t}, r_t)), \\
 (3) \quad & m_t = \sum_{s=1}^M \alpha_{s,t} u_{s,t}.
 \end{aligned}$$

For semantics v2, the RWKV decay is $d_t = \exp(-\exp(w_t))$, and the learned beta gate controls the full write/correction pair instead of introducing a second carry decay. The candidate selected-slot update can be written schematically as

$$(4) \quad \tilde{S}_{j_t,t+1} = d_t \odot S_{j_t,t} + \beta_t \odot (v_t k_t^\top) + \eta \beta_t \odot ((S_{j_t,t} a_t) b_t^\top),$$

where η is the configured RWKV correction multiplier; the exact beta/erase/keep wiring is versioned in the runtime. The correction direction is structural: the implementation forms $a_t = -\hat{k}_t$ and $b_t = \hat{k}_t \odot \text{gate}_t$, so the final plus sign in Equation (4) contains the key-direction subtraction. Only slot j_t is replaced; all other slots persist. The core then applies its bias-free empty-state readout and learned g_t gate to m_t .

frozen hidden stream $\longrightarrow$ normalized low-rank source $\longrightarrow$ RWKV-7 features
 $\longrightarrow M$ persistent FP32 matrix states $\longrightarrow$ cosine-routed read-before-write
 $\longrightarrow$ low-rank $\Delta q, \Delta o$ corrections $\longrightarrow$ frozen attention output

FIGURE 1. The learned path is small and recurrent; the Gemma backbone remains frozen. The best reported Gemma configuration activates only the query and output corrections.

2.2. Frozen-backbone integration. Figure 1 summarizes the production path. The memory read is projected into low-rank corrections for attention heads, extending the online-adaptor setting studied by δ -mem [3]. Although the implementation supports q, k, v, o , the best Gemma run uses only q and o , with scale $\alpha_{\text{adapter}}/d$. Gemma’s original parameters remain frozen. Reported trainable memory sizes are 257,744 parameters for two rank-8 layers, 797,808 for six rank-8 layers, and 1,594,080 for 24 eligible rank-4 layers. Online state is explicitly reset between independent rows and is synchronized with cached decoding within a session.

3. CONTROLLED PROXY EXPERIMENTS

The controlled run is CPU-only and training-free. Its Log-Linear Attention check executes a repository smoke path [4]; the other comparisons are specialized. The RWKV proxy uses direct synthetic keys and values, decay 0.98, and correction scale 1.0; the delta-rule and linear baselines retain their own updates. Block comparisons construct matrices over supplied blocks and read the known query block; single-state baselines process the full stream. The suite compares fixed and adaptive partitions, self-contained delta-rule and RWKV-7 updates, and an exact same-boundary ablation, without exercising the production cyclic writer or cosine all-slot router. Tables report five-seed means; the repository retains per-trial JSONL.

TABLE 1. Controlled test matrix.

Test family	Configurations	Seeds	What it isolates
Log-Linear smoke	1	–	Real repository recurrence executes on CPU
Deviation bound	4	5	Adaptive versus fixed block summarization
Needle recall	3	5	State count and supplied-boundary quality
Same-boundary ablation	15	5	RWKV-7 update versus linear block state

3.1. Adaptive boundaries at matched budget. The Dynamic Linear Attention (DLA) comparison [5] is included because the multi-state RWKV baseline is evaluated on the same state-allocation problem. The reported quantity is the deviation bound from the DLA theorem; lower is better. DLA lowers it in all four configurations, including the capacity-bounded regime where the number of semantic segments exceeds the state budget.

TABLE 2. Deviation bound at matched state budget, mean over five seeds.

Segments	K	Fixed	DLA	Reduction
6	8	45.898	23.839	48.1%
10	12	58.908	26.054	55.8%
16	8	143.390	118.131	17.6%
24	10	193.717	169.493	12.5%

3.2. Associative recall across state allocations. The synthetic study mixes rare “needle” key/value pairs with redundant filler and measures cosine similarity between the retrieved vector and the target value. Table 3 reports means over five seeds. The fixed, DLA, and multi-state columns use the same number of block states shown in the table. Multi-state RWKV applies the specialized RWKV-7 update token by token within each DLA block; the single-state RWKV comparator intentionally places the entire stream in one matrix. The comparison with that one-state baseline therefore changes memory capacity as well as allocation and does not isolate the recurrence.

TABLE 3. Synthetic associative recall, mean cosine score over five seeds.

Needles	Filler	K	States	Fixed	Single RWKV	Multi-state RWKV	DLA
6	8	16	12	0.920	0.797	1.000	1.000
10	6	24	20	0.934	0.682	1.000	1.000
8	10	20	16	0.887	0.626	1.000	1.000

The clean conclusion is comparative: multiple states eliminate the interference observed in the single recurrent state on these tested streams. DLA also reaches 1.000 because its adaptive segmentation allocates states well. This table does not distinguish whether performance comes from boundary selection or state dynamics, so the repository also fixes identical token blocks for both methods. Across 15 such cells, the RWKV-7 state wins 11, ties four, and loses none against the linear block state. Oracle boundaries yield 1.000 for both; in the hardest listed low- K condition (16 needles, $K = 12$, six states), linear memory scores 0.371 and RWKV-7 scores 0.669. The supported interpretation is that DLA’s main advantage is adaptive state allocation, while RWKV-7’s advantage appears in the update when boundaries are held fixed and imperfect.

TABLE 4. RWKV-7 minus linear recall under identical boundaries, mean over five seeds.

Needles / filler / K	Oracle	DLA	Fixed	Noisy DLA	Low- K DLA
8 / 12 / 16	+0.000	+0.000	+0.133	+0.112	+0.313
12 / 10 / 16	+0.000	+0.190	+0.228	+0.282	+0.373
16 / 8 / 12	+0.000	+0.272	+0.324	+0.311	+0.299

3.3. Surprise-gated exact cache. The HOLA extension [6] uses the same externally blocked synthetic proxy as a compressive “neocortex” and adds a bounded exact-key/value cache admitted by write magnitude. On the hardest 16-needle low- K cell, a 16-entry surprise cache raises recall from 0.669 to 0.880; a matched recency cache gives 0.665 and a flat-read control gives 0.683. Results are means over five seeds. The gain therefore depends on informative admission and selective readout, not simply retaining the most recent keys. This remains a synthetic proxy experiment rather than a learned language-model result.

The full grid contains 30 mean rows: cache width $w \in \{8, 16\}$, three needle/filler settings, and five boundary policies, each averaged over five seeds. The registered hypothesis checks are: surprise versus recency, mean gap +0.038; sharp versus flat read, mean gap +0.031 (flat versus no cache +0.003); recovery of the low- K oracle gap,

0.55; the same HOLA cache atop its GDN versus RWKV-7 backbone, mean absolute difference 0.0005; and, in the width-16 bad-boundary aggregate, a positive cache gain for delta-rule-like backbones (+0.053 versus -0.016 for a plain linear state). A raw surprise gate was not sufficient with an untrained constant gate; the reported consolidated result uses online consolidation and a read-confidence gate.

4. LEARNED GEMMA4 EXPERIMENTS

4.1. Tau2 application screen. The accepted comparison uses solo/dummy-user mode, greedy decoding, seed 300, no evaluation-time mobile-data rule planner, and no parser format-repair patch. Table 5 shows selected runs from the 20-task telecom screen. The best checkpoint is a six-layer rank-8 q, o adapter at step 100 of a format-refresh continuation.

TABLE 5. Accepted no-rule Tau2 screen. Pass@1 is the single-run task-success count; each result contains only 20 tasks.

Path	Layers/rank	Pass@1
Frozen Gemma4 E4B baseline	none	4/20 (0.20)
Original 82-row online-memory data, 656 updates	0-1/r8	1/20 (0.05)
Generated action SFT, 656 updates	0-5/r8	10/20 (0.50)
All eligible layers, generated action SFT	0-23/r4	1/20 (0.05)
Format-refresh continuation, final checkpoint	0-5/r8	12/20 (0.60)
Format-refresh continuation, step 100	0-5/r8	14/20 (0.70)

The screen is promising but fragile. The final checkpoint from the same continuation falls from 14/20 to 12/20, revealing checkpoint-selection sensitivity. Wider placement performs poorly despite lower training loss, and the original 82-row data does not transfer. At $n = 20$, a larger confirmation run (at least 50 tasks, ideally the full telecom split) is required before treating the apparent gain over 4/20 as robust.

4.2. Does the identity of state control the answer? Scene-memory experiments progressively separate fitting, state presence, and state identity. A 24-layer semantics-v2 run reaches in-sample CE 1.626 after 672 updates on 32 rows, but that is memorization evidence only. A later correct-versus-shuffled objective obtains a shuffled-minus-correct full-answer CE gap of only +0.00268. V16 improves a narrow in-sample first-token donor-minus-correct gap to +0.07031, yet attribution validation is 16/30, below the frozen base at 25/30, and the remaining benchmark is incomplete.

The post-V16 32-row scene pilot provides the clearest causal controls. Its primary metric is recovered scene-boundary micro-F1:

TABLE 6. Post-V16 scene pilot at step 128. “Donor” substitutes another row’s state; “zero” disables stored state.

Condition	Base	Normal	No-write	State-only	Donor	Zero
Recovered micro-F1	0.1379	0.1868	0.1379	0.0851	0.0851	0.0000

Table 6 shows that normal minus no-write is +0.0489, and state-only minus zero is +0.0851: learned writes and nonzero recurrent state affect prediction. However, correct and donor state both score 0.0851. The experiment therefore establishes a state-presence effect, not useful history-identity retrieval. The rows are exact-row and exact-prompt disjoint from training, but passage-level semantic overlap was not fully audited; no test split or full validation run was completed.

5. RUNTIME AND VERIFICATION

The maintained Hugging Face path bundles state attachment, cached decoding, reset, serialization, and training support. Tests cover recurrent projection and update math, eager and SDPA Gemma attention, shared-KV behavior, cached decoding, write-disable and donor/shuffled controls, runtime state save/load, scene data contracts, evaluator gates, and slot-route tracing. Reported milestones are versioned rather than additive: the RWKV initialization fix reports 59 regression tests; the completed scene pilot reports 169 core, 128 trainer/snapshot/resume,

and 87 preparation/evaluator checks followed by a 460-test repository run; the later protected candidate reports 111 focused evaluator tests and 666 repository-wide tests.

The GGUF work exports a compact RWKV-MS sidecar with an isolated projection, read-before-write, read-out, and active q, o -delta fixture. A patched local Gemma4 runtime consumes it, synchronizes mutable state, saves/restores sessions and server slots, and rejects corrupt restores. The port remains experimental: its best-tested path supports one sequence, one state head, q, o deltas, and serial physical microbatches ($-ub\ 1$). Multi-token prompt graphs exist, but stronger state-level parity is still required; the fixture does not establish complete end-to-end equivalence. Table 7 records the completed checks.

TABLE 7. Completed runtime and portability checks recorded by the project.

Check	Recorded result	Evidence established
Focused Qwen/Gemma/RWKV regressions	78 passed	Changed Python runtime paths execute
Gemma4 E4B A800 train smoke	2 updates; 21 tensors changed	Nonzero state/gradient and exact reload
GGUF sidecar manifest	186 BF16 tensors	1,595,616 tensor bytes; six q, o layers
Checkpoint sidecar round trip	797,808 parameters; exact	All 186 tensors reconstruct
PyTorch golden math fixture	max abs. diff 0.0	37 tensor records: projection/scan/readout
llama.cpp C++ fixture	51 values; 1.37×10^{-6}	Sidecar-driven C++ math parity
First generated-text trace	exact match	One deterministic prompt/output smoke

6. LIMITATIONS AND NEXT TESTS

The following boundaries are material to interpreting the results.

- Proxy recall uses designed vectors and fixed coefficients; block variants use supplied blocks and known block-index reads. It omits learned projections, the cyclic writer, and cosine all-slot routing.
- Few trainable parameters can change frozen-backbone behavior, but this small screen cannot establish broad memory learning or unrelated-capability retention.
- The Tau2 best checkpoint is selected on 20 tasks and degrades with further training.
- Scene controls show write and state-presence effects, while correct-state superiority over donor or shuffled state has not generalized.
- Earlier scene splits are row-disjoint but not fully passage-disjoint; the repository records substantial paragraph overlap in the newer candidate audit.
- The GGUF runtime is a constrained systems prototype, not a production multi-session implementation.

Decisive next tests are a larger fixed Tau2 set; passage-disjoint memory data; held-out scenes where correct state beats donor, shuffled, zero, and no-write controls; and state-level parity across multi-token scans. A learned online-memory claim should pass every causal and held-out gate, not merely reduce loss or match same-row logits.

7. ARTIFACT LEDGER

Numerical and implementation anchors appear below; see also the source repository and released adapter.

Topic	Repository artifact
Synthetic recall and boundary ablations	EVAL.md; dla_poc.py
HOLA extension	experiments/hola_hippocampus/REPORT.md
Tau2 screen and parameter counts	GEMMA_RWKV_MS_TAU2_TRAINING_PLAN_V2.md
Scene versions and causal controls	experiments/rethinking_rwkv_ms_gemma/VERSION_PROGRESSION_REPORT.md
Scene acceptance criteria	experiments/rethinking_rwkv_ms_gemma/SCENE_HARD_FAILURE_EXPERIMENT_PLAN.md
RWKV projection and recurrence	deltamem/core/hrm_rwkv7.py; deltamem/core/delta_impl.py
GGUF port status and constraints	GGUF_EXTERNAL_MEMORY_FEASIBILITY.md; integrations/delta_mem_rwkv_ms/GGUF_PORT_PLAN.md

REFERENCES

- [1] *Multi-State RWKV Online Memory*, repository documentation and experiment artifacts, snapshot reviewed August 2, 2026.
- [2] Bo Peng et al., “RWKV-7 ‘Goose’ with Expressive Dynamic State Evolution,” arXiv:2503.14456, 2025.
- [3] Jingdi Lei, Di Zhang, Junxian Li, et al., “ δ -mem: Efficient Online Memory for Large Language Models,” arXiv:2605.12357, 2026.
- [4] Han Guo, Songlin Yang, Tarushii Goel, et al., “Log-Linear Attention,” arXiv:2506.04761, 2025.
- [5] Xin Wang, Hui Shen, Boyuan Zheng, et al., “Dynamic Linear Attention,” arXiv:2606.10650, 2026.
- [6] Wanyun Cui, “A Hippocampus for Linear Attention: An Exact Memory for What the Recurrent State Forgets,” arXiv:2607.02303, 2026.